

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

HASHING II



WHERE LECTURE 08 LEFT OFF

ESTABLISHED

A hash table is an array of m slots plus a hash function $h : U \rightarrow \{0, \dots, m-1\}$.

Collisions are forced by pigeonhole, since $|U| > m$. No choice of h avoids them.

The load factor $\alpha = n/m$ is the parameter every bound is stated in.

A good h is deterministic, fast, uniform, and uses the whole key.

Universal hashing puts the randomness in our choice of h , so the bounds survive adversarial keys.

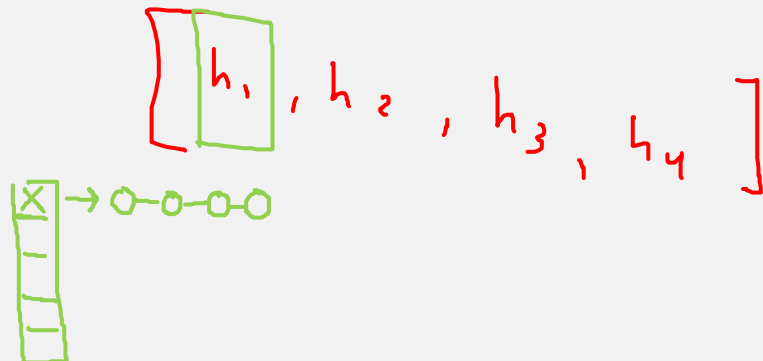
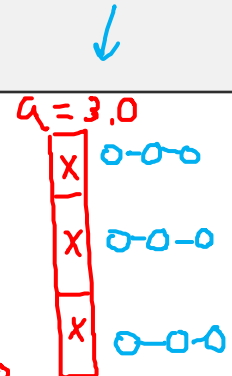
STILL OPEN

When two keys collide, where does the second one actually go?

We asserted expected cost $\Theta(1 + \alpha)$ and never proved it. Prove it.

How does the cost change as the table fills? Is $\alpha = 0.9$ fine, or a disaster?

Can we do better than expected constant time? Can we get worst-case constant time, and what does that cost?

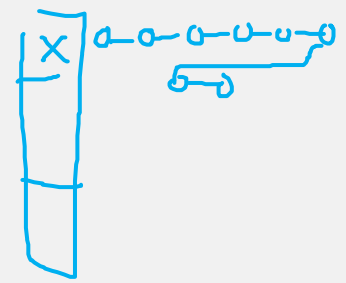


SEPARATE CHAINING



SEPARATE CHAINING: ONE LIST PER SLOT

$$a = \frac{9}{3} = 3$$



The table stores pointers, not keys. Each slot holds a linked list, called a chain, containing every key that hashed to it.

```
insert(k, v):  
    prepend (k,v) to chain[h(k)]    # O(1)  
  
lookup(k):  
    scan chain[h(k)] for key k      # O(len)  
  
delete(k):  
    unlink k from chain[h(k)]       # O(len)
```

M = 7, H(K) = K MOD 7, N = 7, α = 1

SLOT	CHAIN	LENGTH
0	14	1
1	8 → 22 → 22	2
2	empty	0
3	17	1
4	11 → 25	2
5	empty	0
6	20	1

set ()
0 (4)

THE COST MODEL

Hashing is $\Theta(1)$. Everything after it is a linear scan of one chain. So the runtime question is entirely: how long is the chain we land in?



EXPECTED COST OF AN UNSUCCESSFUL SEARCH

Searching for a key that is not in the table. We have to scan the entire chain at slot $h(k)$ before we can conclude it is absent, so the cost is exactly the chain length.

Let $X_i = 1$ if key i hashed to slot $h(k)$
0 otherwise

$$E[X_i] = \Pr[X_i = 1] = 1/m \quad (\text{SUHA})$$

$$\begin{aligned} E[\text{chain length}] &= E[\sum_{i=1..n} X_i] \\ &= \sum_{i=1..n} E[X_i] \quad (\text{linearity}) \\ &= n * (1/m) \\ &= \alpha \end{aligned}$$

Handwritten notes: $\alpha \rightarrow h$, $\alpha(1+\alpha)$

$$\begin{aligned} \text{cost} &= 1 \text{ (hash)} + \alpha \text{ (scan)} \\ &= \Theta(1 + \alpha) \end{aligned}$$

Handwritten arrow pointing from $\Theta(1 + \alpha)$ to the next box.

WHY LINEARITY IS ENOUGH

Linearity of expectation holds for any random variables, independent or not. That is why this proof is three lines instead of a page.

THE +1 IS NOT DECORATION

As $\alpha \rightarrow 0$ the cost does not vanish. We still hash and still touch one slot. Writing $\Theta(\alpha)$ instead of $\Theta(1 + \alpha)$ would be wrong.

WHEN THIS IS $\Theta(1)$

Keep m proportional to n . Then α is a constant and $\Theta(1 + \alpha)$ collapses to constant time.

Handwritten notes: const. , $\downarrow$, $\Theta(1)$



EXPECTED COST OF A SUCCESSFUL SEARCH

Searching for a key that is present. On average we stop partway down its chain rather than walking all of it, so the constant improves, but the asymptotic answer does not.

```
# Keys are prepended, so the keys scanned before  
# key i are exactly those inserted after it.
```

Let Y_i = # keys inserted after i that landed
in the same slot as i

$$E[Y_i] = (n - i) / m \quad (\text{SUHA})$$

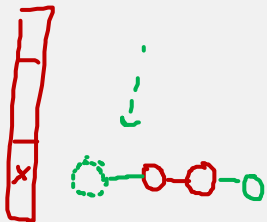
$$\begin{aligned} E[\text{cost}] &= 1 + (1/n) \sum_{i=1..n} (n-i)/m \\ &= 1 + (n - 1) / (2m) \\ &= 1 + \alpha/2 - \alpha/(2n) \\ &= \Theta(1 + \alpha) \end{aligned}$$

SAME BOUND, BETTER CONSTANT

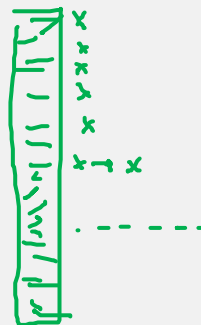
A miss costs about α extra probes; a hit costs about $\alpha/2$. Chaining does not distinguish the two cases asymptotically. Open addressing will. Its two formulas are genuinely different functions.

THE WORST CASE IS UNCHANGED

All of this is expectation. If every key hashes to one slot, the chain has length n and lookup is $\Theta(n)$. SUHA is what rules that out, and universal hashing from last lecture is what makes SUHA safe to rely on against an adversary.



↓
 $O(1)$



RESIZING IS WHAT HOLDS α CONSTANT

C++

array

vector

←---

$\Theta(1 + \alpha)$ is only $\Theta(1)$ if α is bounded. Nothing in the structure enforces that. A resize policy does.

```
insert(k, v):  
    ... normal insert ...  
    n = n + 1  
    → if n / m > LOAD_THRESHOLD:      # e.g. 0.75  
        m = 2 * m  
        rehash all n keys into the new table
```

$O(1) \cdot n$

$O(n)$

AMORTISED ANALYSIS

Doubling means a resize at size n was preceded by resizes at $n/2$, $n/4$, and so on. Total rehashing work through n inserts is $n + n/2 + n/4 + \dots < 2n$, so the amortised cost per insert is $O(1)$.

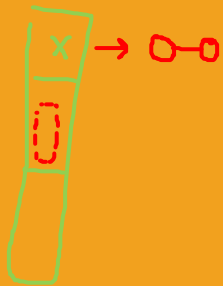
$O(n)$

WHY REHASHING IS UNAVOIDABLE

$h(k)$ depends on m . Changing m changes every key's slot, so the entries cannot be copied across. Each one must be hashed again and re-placed. A single resize costs $\Theta(n)$.

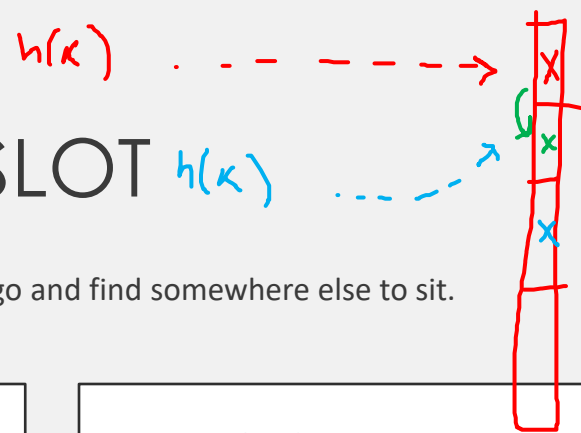


OPEN ADDRESSING



PROBE UNTIL YOU FIND A FREE SLOT

No lists and no pointers. Every key lives in the table array itself, so a colliding key has to go and find somewhere else to sit.



THE PROBE SEQUENCE

The hash function gains a second argument. $h(k, i)$ is the i -th slot to try, for $i = 0, 1, 2, \dots$, and the sequence $\langle h(k, 0), \dots, h(k, m-1) \rangle$ should be a permutation of all m slots.

WHAT YOU GAIN

One contiguous array, no pointer overhead, and far better cache locality than chasing linked-list nodes through memory.

```
insert(k, v):
    for i in 0, 1, ..., m-1:
        j = h(k, i)
        if table[j] is empty:
            table[j] = (k, v); return
    raise TableFull
```

```
lookup(k): probe the same sequence; stop at k,
           or at the first EMPTY slot
```

WHAT YOU GIVE UP

$\alpha \leq 1$ is now mandatory. The table cannot hold more keys than it has slots.

THE INVARIANT EVERYTHING RESTS ON

Reaching an empty slot proves k is absent. If k were present, insert would have put it there. This is exactly what deletion is about to break.



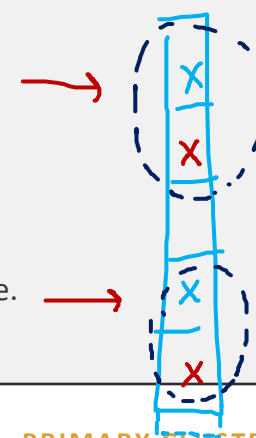
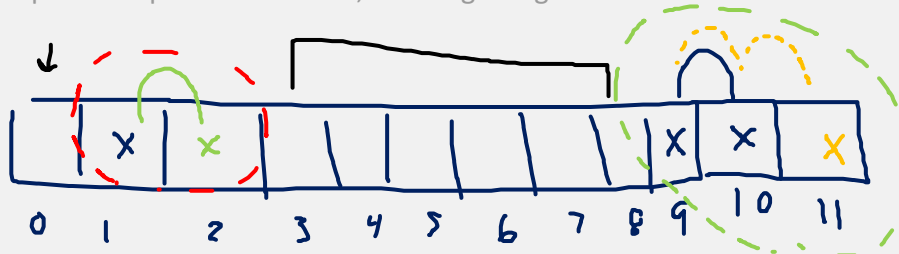
LINEAR PROBING

$h(k, i) = (h(k) + i) \bmod m$. On a collision, walk forward one slot at a time and wrap around at the end of the table.

$M = 11$, $H(K) = K \bmod 11$ INSERT 20, 34, 45, 31, 42

KEY	H(K)	PROBE SEQUENCE	LANDS	PROBES
20	9	9 free	9	1
34	1	1 free	1	1
45	1	1 taken → 2 free	2	2
31	9	9 taken → 10 free	10	2
42	9	9, 10 taken → 0 free	0	3

Final table: slot 0 = 42, slot 1 = 34, slot 2 = 45, slot 9 = 20, slot 10 = 31. Note that 42 wrapped past the end of the table back to slot 0, since the probe sequence is circular, and forgetting that is the most common trace error on exams.



PRIMARY CLUSTERING

Occupied slots coalesce into long contiguous runs. A cluster of length L is hit by any key hashing anywhere inside it, so it grows with probability proportional to L , and the longer it gets, the faster it grows.

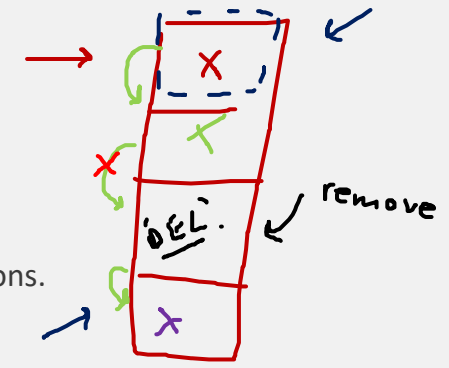
The result is a rich-get-richer feedback loop. A few very long runs form, and probe counts degrade far faster than the load factor alone would suggest.



DELETION BREAKS THE PROBE SEQUENCE

$h(x)$ $K \bmod 10$

10
20 x
30 x
40 x



Under chaining, deletion is an unlink and there is nothing to think about. Under open addressing it is the hardest of the three operations.

drop n a

THE BUG

Lookup stops at the first empty slot. If deletion empties a slot in the middle of some key's probe sequence, every key beyond that point becomes unreachable. It is still stored and still occupying memory, but reported as absent by every future lookup.

CONCRETELY

In the table from the last slide, delete 45 from slot 2 by emptying it.

Now look up any key whose probe sequence passes through slot 2. It halts there and reports the key absent, even though the key is sitting in slot 3 or 4, exactly where insert put it.

THE FIX: TOMBSTONES

Mark the slot DELETED rather than EMPTY. Lookup treats DELETED as occupied and keeps probing; insert treats it as free and may reuse it.

The cost is that tombstones accumulate, so search times keep degrading even as n falls. Periodic rehashing is the only way to clear them.

This is the strongest practical argument for chaining. A delete-heavy workload on an open-addressed table degrades steadily until it is rebuilt, and the load factor gives you no warning that it is happening.



QUADRATIC PROBING

$h(k, i) = (h(k) + c_1i + c_2i^2) \bmod m$. Below, the common form with $c_1 = 0$ and $c_2 = 1$, so the offsets are 0, 1, 4, 9, 16, ...

SAME KEYS: M = 11, INSERT 20, 34, 45, 31, 42

KEY	H(K)	PROBE SEQUENCE (OFFSET i^2)	LANDS	PROBES
20	9	+0 → 9 free	9	1
34	1	+0 → 1 free	1	1
45	1	+0 → 1, +1 → 2 free	2	2
31	9	+0 → 9, +1 → 10 free	10	2
42	9	+0 → 9, +1 → 10, +4 → 2, +9 → 7	7	4

Under linear probing, 42 landed in slot 0. Here it lands in slot 7. The offsets 0, 1, 4, 9 from base 9 give slots 9, 10, 2, 7. Recall that $13 \bmod 11 = 2$ and $18 \bmod 11 = 7$.

SECONDARY CLUSTERING

Primary clustering is gone. Probes jump away instead of filling a run. But keys sharing the same $h(k)$ still follow identical probe sequences, so they still pile up on each other.

THE TERMINATION GUARANTEE

If m is prime and $\alpha < 1/2$, this scheme is guaranteed to find an empty slot: the first $\lceil m/2 \rceil$ probes land on distinct slots. Without that condition, probing can cycle forever while free slots remain.

